Ecole Supérieure
d'Informatique et du Numérique
COLLEGE OF ENGINEERING & ARCHITECTURE

COMPTE RENDU

Cyber Sécurité — TP6 — Introduction à iptables & Firewalls
3e année Cybersécurité — École Supérieure d'Informatique et du Numérique (ESIN)
Collège d'Ingénierie & d'Architecture (CIA)

Étudiants :	HATHOUTI Mohammed Taha JIDAL Ilyas KABORE Mohammad Sharif Jonathan
Filière :	Cybersécurité
Année :	2025/2026
Enseignant :	M.SEBBAR & Mme.GADI
Date :	30 mars 2026

Table des matières

Objectifs **2**
Précautions Légales et Éthiques 2

Configuration **2**

1 Mise en place du client et du serveur **3**
1.1 Configuration réseau et vérification de la connectivité 3
1.2 Démarrage du serveur Apache 3
1.3 Démarrage du serveur SSH 5

2 Analyse des ports ouverts **6**
2.1 Scan Nmap depuis le client Windows 7 6
2.2 Vérification côté serveur avec netstat 6

3 Configuration du pare-feu iptables **7**
3.1 Présentation d'iptables 7
3.2 Création du script de configuration 7

4 Question 2 — Test avec la politique INPUT DROP **8**
4.1 Vérification des règles iptables 8
4.2 Test 2-1 : Ping depuis Windows 7 9
4.3 Test 2-2 : Scan Nmap depuis Windows 7 9
4.4 Test 2-3 : Navigation HTTP depuis Windows 7 9

5 Ajout des règles d'autorisation **10**
5.1 Règle 1 : Loopback et connexions établies 10
5.2 Règle 2 : Autorisation du ping (ICMP) 10
5.3 Règle 3 : Autorisation du trafic HTTP (port 80) 10
5.4 Règle 4 : Autorisation du SSH depuis le client uniquement 11
5.5 Scan Nmap final (après toutes les règles) 12

6 Synthèse **12**
6.1 Récapitulatif des règles iptables 12
6.2 Résultats des tests avant et après configuration 12
6.3 Impact sur la Triade CIA 13

Conclusion **13**

Objectifs

L'objectif de ce TP est d'installer et de configurer un pare-feu sur une machine Linux afin de séparer deux segments réseau différents et d'implémenter une politique de sécurité adaptée. Les étapes abordées sont les suivantes :

- Configuration des paramètres réseau d'un serveur web (Ubuntu) et d'une machine cliente (Windows 7) ;
- Utilisation de Nmap pour l'analyse des ports ouverts ;
- Sécurisation du serveur web par la configuration du pare-feu iptables ;
- Application et test de règles de filtrage sur les chaînes INPUT et OUTPUT ;

Précautions Légales et Éthiques

IMPORTANT : Les techniques de configuration de pare-feu et d'analyse réseau présentées dans ce TP sont réalisées dans un environnement isolé, sur deux machines physiques connectées via un routeur Huawei **sans accès Internet**, dans un cadre pédagogique contrôlé.

Configuration

Ce TP a été réalisé sur un réseau local isolé (routeur Huawei, connexion Ethernet, sans Internet) avec les deux machines suivantes :

- **Serveur (Ubuntu 24.04)** : `hathouti@Taha-HP-ProBook` — IP : **192.168.1.20**, interface `enp0s31f6`
- **Client (Windows 7)** : `TAHA-PC` — IP : **192.168.1.10**

Note : Les interfaces Wi-Fi des deux machines restent connectées à Internet pour permettre les échanges extérieurs. Le trafic du TP transite exclusivement via les interfaces Ethernet connectées au routeur Huawei.

1 Mise en place du client et du serveur

1.1 Configuration réseau et vérification de la connectivité

Les deux machines sont connectées au routeur Huawei via câble Ethernet. Les adresses IP ont été vérifiées avec les commandes suivantes :

Listing 1 – Vérification de l'IP du serveur Ubuntu

```
ip a show enp0s31f6
```

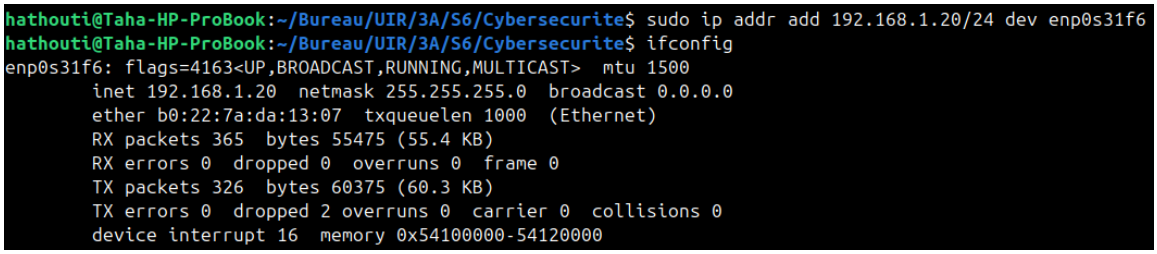


FIGURE 1 – Interface `enp0s31f6` du serveur Ubuntu — IP : 192.168.1.20/24

La connectivité entre les deux machines a été vérifiée par un ping bidirectionnel :

Listing 2 – Ping du serveur Ubuntu vers le client Windows 7

```
ping -c 4 192.168.1.10
```

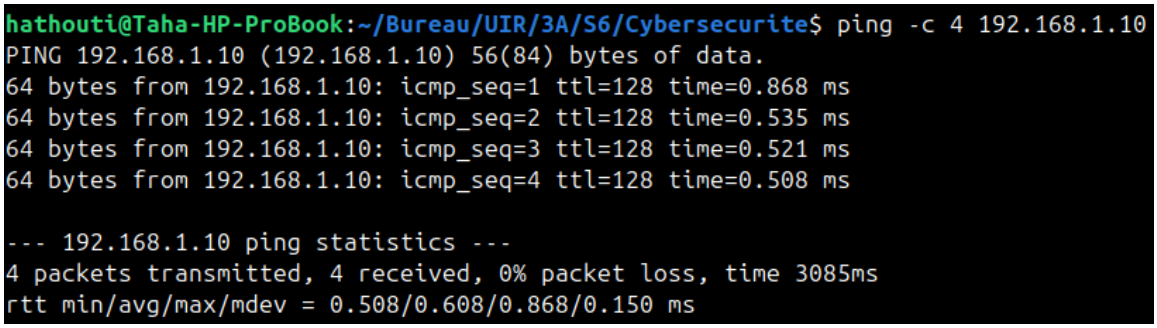


FIGURE 2 – Ping Ubuntu → Windows 7 : 4 paquets transmis, 0% de perte

1.2 Démarrage du serveur Apache

Le serveur web Apache2 a été installé et démarré sur la machine Ubuntu :

Listing 3 – Installation et démarrage d'Apache2

```
sudo apt-get install apache2
sudo service apache2 start
sudo service apache2 status
```

```
hathouti@Taha-HP-ProBook:~/Bureau/UIR/3A/S6/Cybersecurite$ sudo service apache2 start
hathouti@Taha-HP-ProBook:~/Bureau/UIR/3A/S6/Cybersecurite$ sudo service apache2 status
● apache2.service - The Apache HTTP Server
     Loaded: loaded (/usr/lib/systemd/system/apache2.service; enabled; preset: enabled)
     Active: active (running) since Mon 2026-03-30 21:24:38 +01; 44min ago
       Docs: https://httpd.apache.org/docs/2.4/
   Main PID: 6315 (apache2)
      Tasks: 55 (limit: 18617)
     Memory: 6.1M (peak: 7.0M)
        CPU: 221ms
     CGroup: /system.slice/apache2.service
             ├─6315 /usr/sbin/apache2 -k start
             ├─6318 /usr/sbin/apache2 -k start
             └─6319 /usr/sbin/apache2 -k start
```

FIGURE 3 – Apache2 — Service actif (running) depuis le 30 mars 2026

La page par défaut d'Apache a été vérifiée depuis le navigateur du serveur :

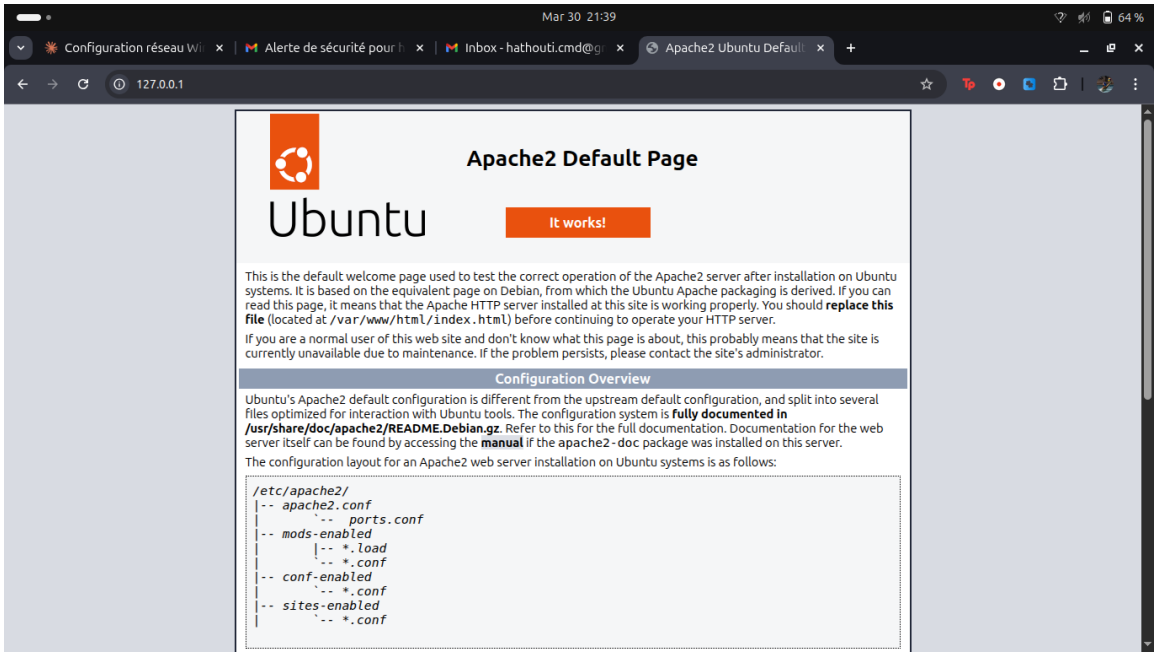


FIGURE 4 – Page Apache2 par défaut accessible depuis le serveur Ubuntu via `http://127.0.0.1`

Et depuis le navigateur du client Windows 7 :

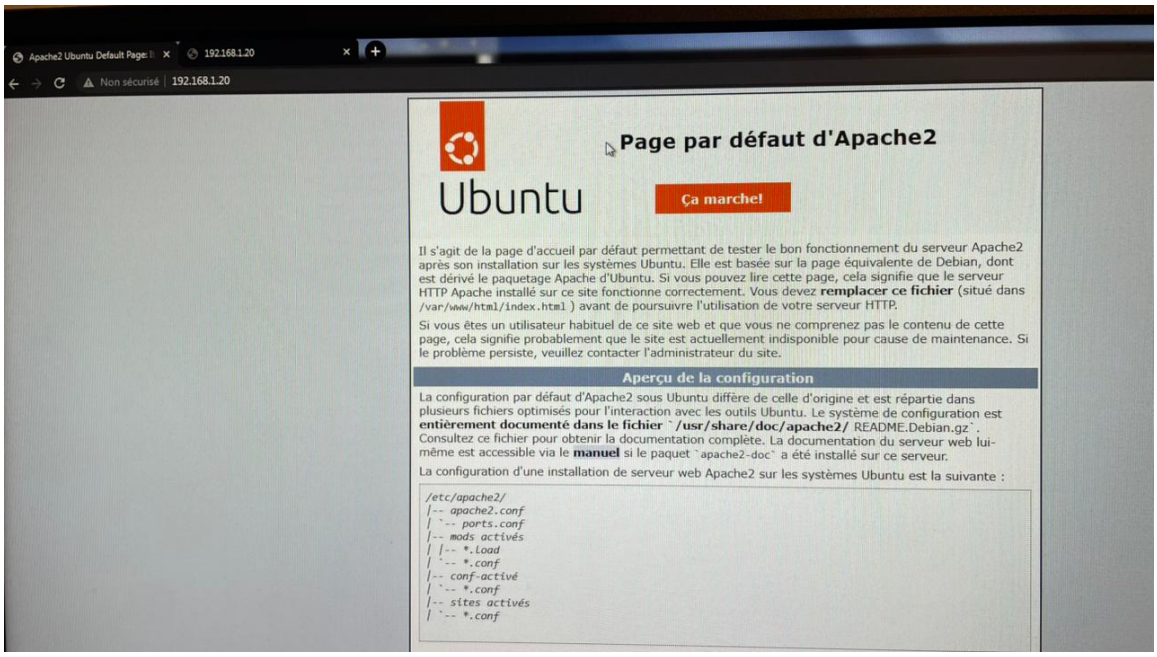


FIGURE 5 – Page Apache2 accessible depuis le client Windows 7 via `http://192.168.1.20`

1.3 Démarrage du serveur SSH

Le serveur OpenSSH a été installé et démarré sur la machine Ubuntu :

Listing 4 – Installation et démarrage du serveur SSH

```
sudo apt-get install openssh-server
sudo service ssh start
sudo service ssh status
```

```
hathouti@Taha-HP-ProBook:~/Bureau/UIR/3A/S6/Cybersecurite$ sudo service ssh start
hathouti@Taha-HP-ProBook:~/Bureau/UIR/3A/S6/Cybersecurite$ sudo service ssh status
● ssh.service - OpenBSD Secure Shell server
     Loaded: loaded (/usr/lib/systemd/system/ssh.service; disabled; preset: enabled)
     Active: active (running) since Mon 2026-03-30 21:42:00 +01; 27min ago
TriggeredBy: ● ssh.socket
       Docs: man:sshd(8)
             man:sshd_config(5)
    Process: 26858 ExecStartPre=/usr/sbin/sshd -t (code=exited, status=0/SUCCESS)
   Main PID: 26859 (sshd)
      Tasks: 1 (limit: 18617)
     Memory: 1.2M (peak: 1.7M)
        CPU: 23ms
     CGroup: /system.slice/ssh.service
             └26859 "sshd: /usr/sbin/sshd -D [listener] 0 of 10-100 startups"
```

FIGURE 6 – SSH — Service actif (running), écoute sur le port 22 (0.0.0.0 et : :)

2 Analyse des ports ouverts

2.1 Scan Nmap depuis le client Windows 7

Avant toute configuration du pare-feu, un scan Nmap a été effectué depuis le client Windows 7 afin d'identifier les ports ouverts sur le serveur Ubuntu :

Listing 5 – Scan Nmap depuis Windows 7 (avant firewall)

```
nmap 192.168.1.20
```

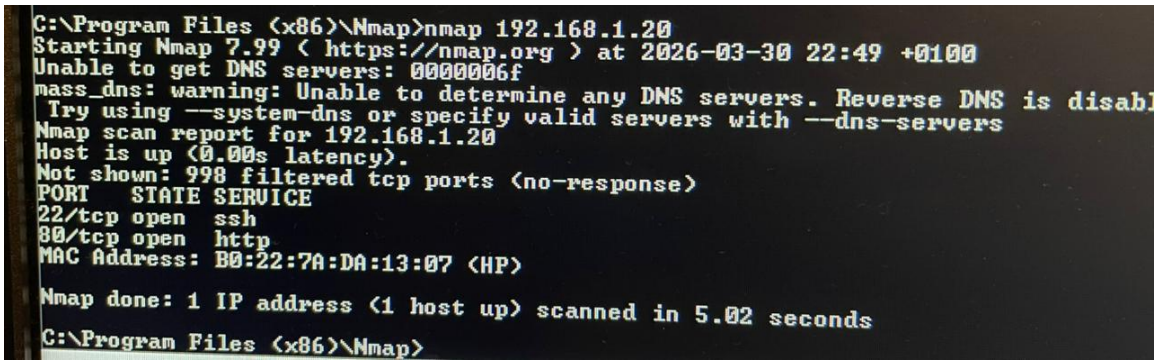


FIGURE 7 – Scan Nmap depuis Windows 7 — Ports 22 (SSH) et 80 (HTTP) ouverts sur 192.168.1.20

Analyse : Nmap détecte deux ports ouverts sur le serveur Ubuntu :

— **Port 22/tcp** : service SSH (openssh)

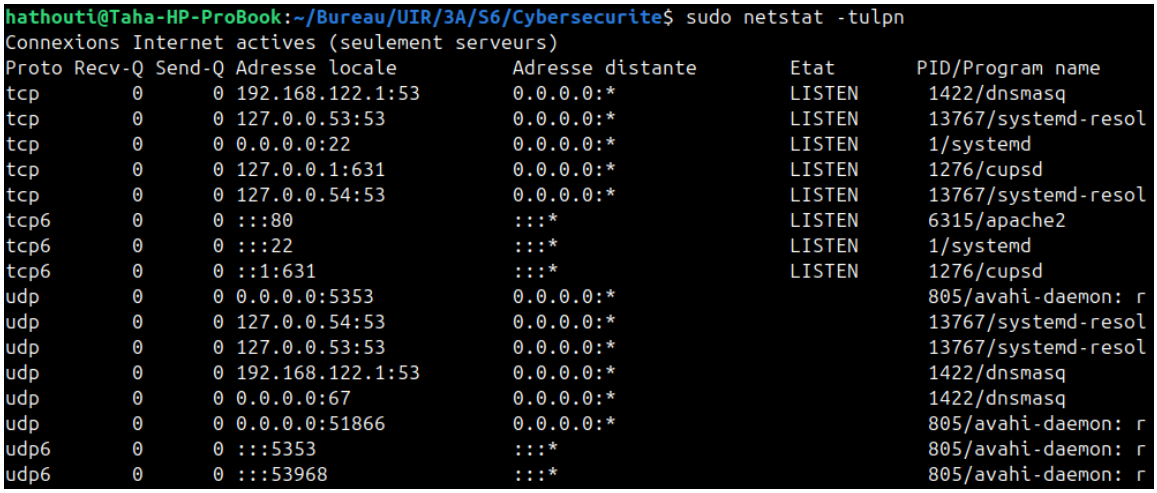
— **Port 80/tcp** : service HTTP (Apache2)

2.2 Vérification côté serveur avec netstat

La commande `netstat` permet de confirmer côté serveur les services en écoute :

Listing 6 – Liste des ports en écoute sur le serveur Ubuntu

```
sudo netstat -tulpn
```



```
hathouti@Taha-HP-ProBook:~/Bureau/UIR/3A/S6/Cybersecurite$ sudo netstat -tulpn
Connexions Internet actives (seulement serveurs)
Proto Recv-Q Send-Q Adresse locale          Adresse distante        Etat       PID/Program name
tcp        0      0 192.168.122.1:53        0.0.0.0:*               LISTEN      1422/dnsmasq
tcp        0      0 127.0.0.53:53           0.0.0.0:*               LISTEN      13767/systemd-resol
tcp        0      0 0.0.0.0:22              0.0.0.0:*               LISTEN      1/systemd
tcp        0      0 127.0.0.1:631           0.0.0.0:*               LISTEN      1276/cupsd
tcp        0      0 127.0.0.54:53           0.0.0.0:*               LISTEN      13767/systemd-resol
tcp6       0      0 :::80                   :::*                    LISTEN      6315/apache2
tcp6       0      0 :::22                   :::*                    LISTEN      1/systemd
tcp6       0      0 ::1:631                 :::*                    LISTEN      1276/cupsd
udp        0      0 0.0.0.0:5353            0.0.0.0:*                           805/avahi-daemon: r
udp        0      0 127.0.0.54:53           0.0.0.0:*                           13767/systemd-resol
udp        0      0 127.0.0.53:53           0.0.0.0:*                           13767/systemd-resol
udp        0      0 192.168.122.1:53        0.0.0.0:*                           1422/dnsmasq
udp        0      0 0.0.0.0:67              0.0.0.0:*                           1422/dnsmasq
udp        0      0 0.0.0.0:51866           0.0.0.0:*                           805/avahi-daemon: r
udp6       0      0 :::5353                 :::*                                805/avahi-daemon: r
udp6       0      0 :::53968                :::*                                805/avahi-daemon: r
```

FIGURE 8 – `netstat -tulpn` — Ports 22 (sshd) et 80 (apache2) confirmés en écoute

Ports identifiés :

— `0.0.0.0:22` (tcp) → `sshd` : écoute sur toutes les interfaces

— `:::80` (tcp6) → `apache2` : écoute sur toutes les interfaces IPv4/IPv6

3 Configuration du pare-feu iptables

3.1 Présentation d'iptables

Linux utilise le module noyau **netfilter** pour le filtrage réseau. Iptables est l'interface en ligne de commande permettant de communiquer avec netfilter. Les règles sont regroupées en **chaînes** prédéfinies :

— **INPUT** : filtre les paquets entrants destinés à la machine locale

— **OUTPUT** : filtre les paquets sortants générés par la machine locale

— **FORWARD** : filtre les paquets routés à travers la machine

— **PREROUTING / POSTROUTING** : modification des paquets avant/après routage

Chaque règle peut déclencher une **cible** (target) :

— **ACCEPT** : laisser passer le paquet

— **DROP** : rejeter silencieusement le paquet

— **REJECT** : rejeter le paquet avec notification

— **LOG** : journaliser les informations du paquet

3.2 Création du script de configuration

Un script Bash `config_firewall.sh` a été créé pour centraliser la configuration du pare-feu :

Listing 7 – Script config_firewall.sh — Configuration du pare-feu iptables

```
#!/bin/bash

echo -n "Suppression des regles existantes ... "
iptables -F INPUT
iptables -F OUTPUT
echo "done"

echo -n "Definition des politiques par defaut ... "
iptables -P OUTPUT ACCEPT
iptables -P INPUT DROP
echo "done"

# Autoriser loopback et connexions etablies
iptables -A INPUT -i lo -j ACCEPT
iptables -A INPUT -m state --state RELATED,ESTABLISHED -j ACCEPT

# Autoriser les requetes ping (ICMP echo-request)
iptables -A INPUT -p icmp --icmp-type echo-request -j ACCEPT
```

```

# Autoriser le trafic HTTP (port 80)
iptables -A INPUT -p tcp --dport 80 -j ACCEPT

# Autoriser SSH uniquement depuis le client Windows 7
iptables -A INPUT -p tcp --dport 22 -s 192.168.1.10 -j ACCEPT
```

Le script est rendu exécutable et lancé avec les droits superutilisateur :

```
chmod +x config_firewall.sh
sudo ./config_firewall.sh
```

4 Question 2 — Test avec la politique INPUT DROP

Dans un premier temps, le script a été exécuté sans les règles d'autorisation (uniquement les politiques par défaut : `OUTPUT ACCEPT` et `INPUT DROP`). Les tests suivants ont été effectués depuis le client Windows 7.

4.1 Vérification des règles iptables

```
sudo iptables -L
```

Listing 8 – `iptables -L` — Chaîne INPUT avec politique DROP, chaîne OUTPUT avec politique ACCEPT

```
#!/bin/bash

echo -n "Suppression des regles existantes ... "
iptables -F INPUT
iptables -F OUTPUT
echo "done"

echo -n "Definition des politiques par defaut ... "
iptables -P OUTPUT ACCEPT
iptables -P INPUT DROP
echo "done"

# Autoriser loopback et connexions etablies
iptables -A INPUT -i lo -j ACCEPT
iptables -A INPUT -m state --state RELATED,ESTABLISHED -j ACCEPT

# Autoriser les requetes ping (ICMP echo-request)
iptables -A INPUT -p icmp --icmp-type echo-request -j ACCEPT

# Autoriser le trafic HTTP (port 80)
iptables -A INPUT -p tcp --dport 80 -j ACCEPT

# Autoriser SSH uniquement depuis le client Windows 7
iptables -A INPUT -p tcp --dport 22 -s 192.168.1.10 -j ACCEPT
```

4.2 Test 2-1 : Ping depuis Windows 7

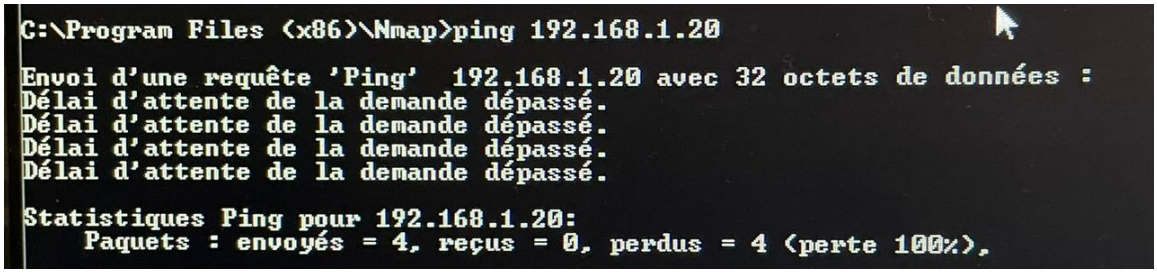


FIGURE 9 – Ping 192.168.1.20 depuis Windows 7 — 100% de perte, délai d'attente dépassé

4.3 Test 2-2 : Scan Nmap depuis Windows 7

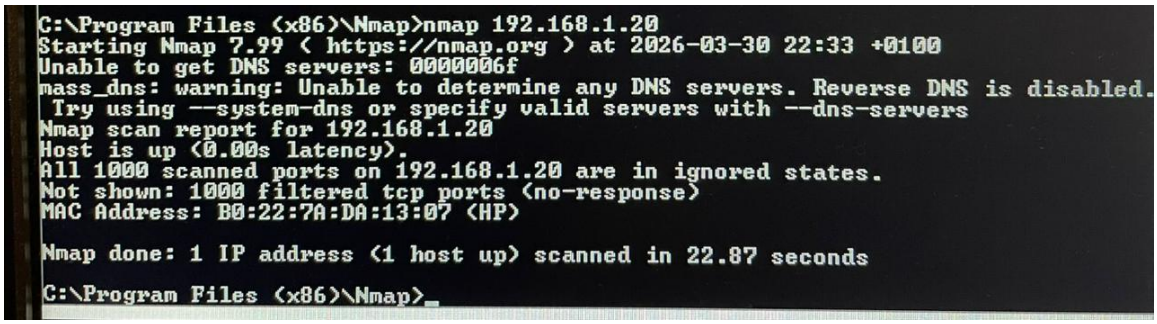


FIGURE 10 – Nmap 192.168.1.20 depuis Windows 7 — 1000 ports filtrés (no-response)

4.4 Test 2-3 : Navigation HTTP depuis Windows 7

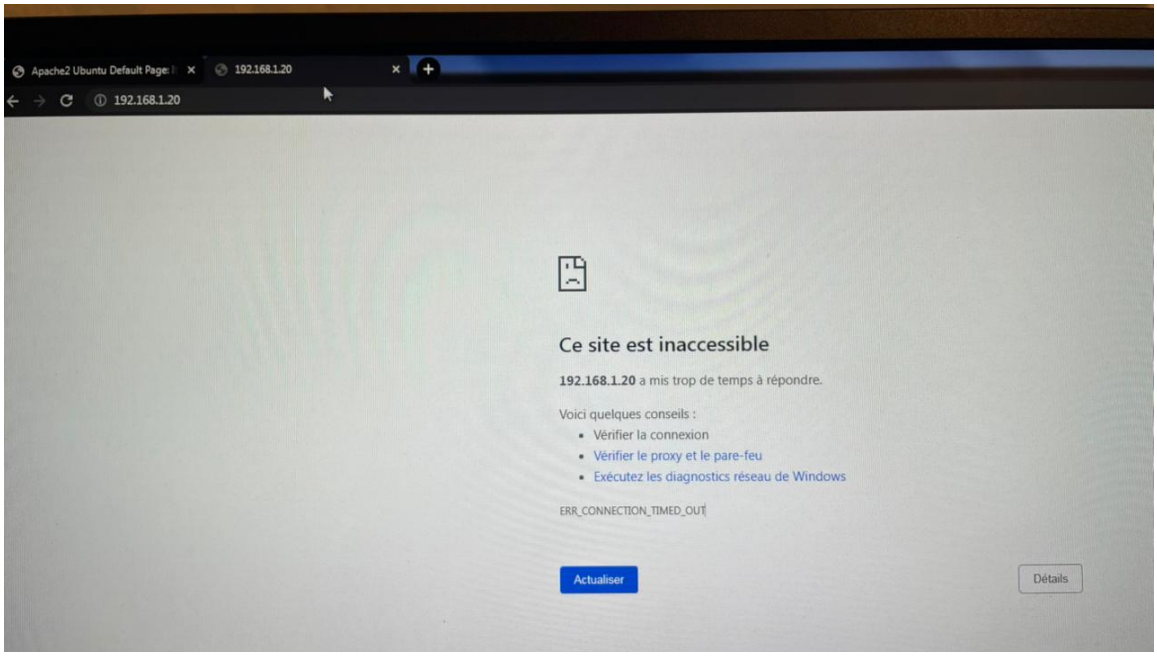


FIGURE 11 – Navigateur Windows 7 — ERR_CONNECTION_TIMED_OUT : site inaccessible

Synthèse Question 2 :

Test	Commande	Résultat
Ping	`ping 192.168.1.20`	Bloqué — 100% perte
Scan de ports	`nmap 192.168.1.20`	Bloqué — 1000 ports filtrés
Navigation HTTP	`http://192.168.1.20`	Bloqué — Connection timed out

TABLE 1 – Résultats des tests avec la politique INPUT DROP (sans règles d'autorisation)

Tout le trafic entrant est bloqué, conformément à la politique `INPUT DROP`. Le serveur est totalement inaccessible depuis le client.

5 Ajout des règles d'autorisation

Le script `config_firewall.sh` a été complété avec les règles d'autorisation spécifiques, puis réexécuté. Les tests ont été répétés à chaque ajout pour observer l'effet de chaque règle.

5.1 Règle 1 : Loopback et connexions établies

```
iptables -A INPUT -i lo -j ACCEPT
iptables -A INPUT -m state --state RELATED,ESTABLISHED -j ACCEPT
```

Ces règles autorisent le trafic sur l'interface de loopback (`lo`) et maintiennent les connexions déjà établies, permettant notamment au serveur de recevoir les réponses à ses propres requêtes sortantes.

5.2 Règle 2 : Autorisation du ping (ICMP)

```
iptables -A INPUT -p icmp --icmp-type echo-request -j ACCEPT
```

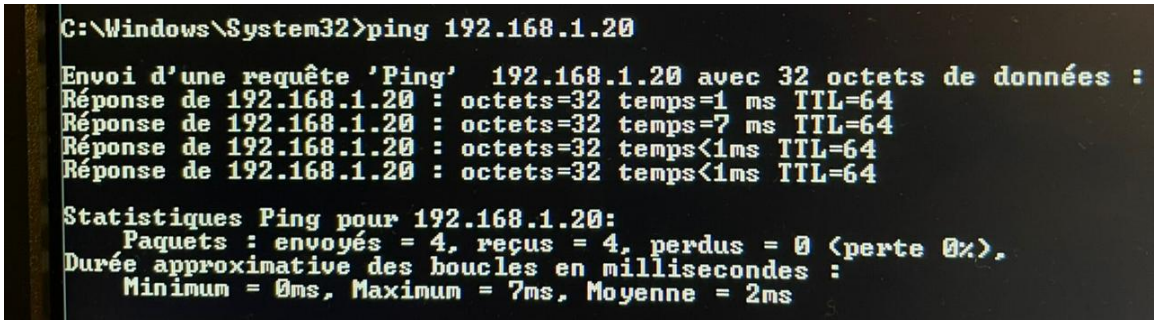


FIGURE 12 – Ping 192.168.1.20 depuis Windows 7 — Réponses reçues, 0% de perte

5.3 Règle 3 : Autorisation du trafic HTTP (port 80)

```
iptables -A INPUT -p tcp --dport 80 -j ACCEPT
```

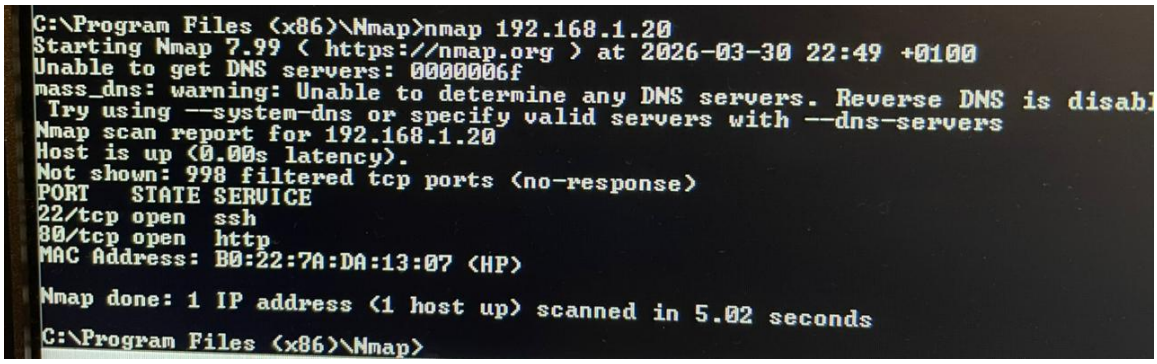


FIGURE 13 – Navigateur Windows 7 — Page Apache2 accessible via http ://192.168.1.20

5.4 Règle 4 : Autorisation du SSH depuis le client uniquement

```
iptables -A INPUT -p tcp --dport 22 -s 192.168.1.10 -j ACCEPT
```

Cette règle autorise les connexions SSH uniquement en provenance de l'adresse IP `192.168.1.10` (le client Windows 7), renforçant ainsi la sécurité en bloquant les tentatives SSH depuis toute autre machine.

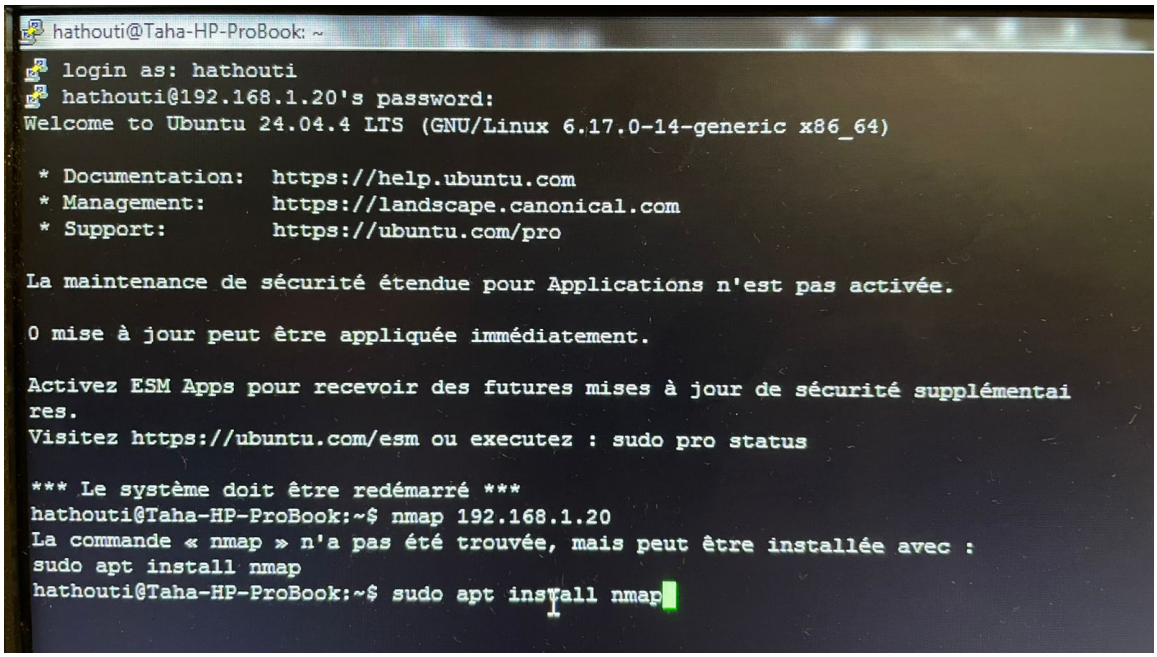


FIGURE 14 – Connexion SSH depuis Windows 7 (PuTTY) vers 192.168.1.20 — Connexion établie

5.5 Scan Nmap final (après toutes les règles)

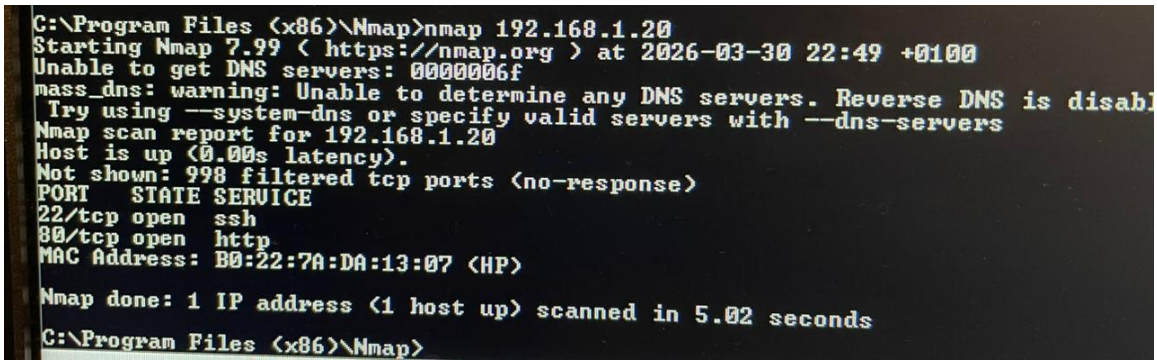


FIGURE 15 – Nmap depuis Windows 7 après les règles iptables — Ports 22 (SSH) et 80 (HTTP) ouverts, reste filtré

Analyse : Après l'ajout des règles, Nmap confirme que seuls les ports 22 et 80 sont accessibles. Les 998 autres ports restent filtrés par la politique `INPUT DROP`, conformément à la politique de sécurité souhaitée.

6 Synthèse

6.1 Récapitulatif des règles iptables

Règle	Commande	Effet
Politique par défaut	`-P INPUT DROP`	Bloquer tout trafic entrant
Politique par défaut	`-P OUTPUT ACCEPT`	Autoriser tout trafic sortant
Loopback	`-A INPUT -i lo -j ACCEPT`	Autoriser le trafic interne
Connexions établies	`-m state --state RELATED,ESTABLISHED`	Maintenir sessions actives
Ping ICMP	`-p icmp --icmp-type echo-request`	Autoriser les pings
HTTP	`-p tcp --dport 80 -j ACCEPT`	Autoriser le web
SSH restreint	`-p tcp --dport 22 -s 192.168.1.10`	SSH depuis client uniquement

TABLE 2 – Récapitulatif des règles iptables configurées

6.2 Résultats des tests avant et après configuration

Test	Avant règles (DROP)	Après règles
Ping	Bloqué	Autorisé
Nmap	1000 ports filtrés	Ports 22 et 80 ouverts
HTTP (port 80)	Connection timed out	Page Apache accessible
SSH (port 22)	Bloqué	Connexion établie (src 192.168.1.10)
SSH (autre IP)	Bloqué	Toujours bloqué (filtrage par source)

TABLE 3 – Comparaison des résultats avant et après configuration du pare-feu

6.3 Impact sur la Triade CIA

Propriété CIA	Impact du pare-feu
Confidentialité	Renforcée : SSH restreint à une seule IP source
Intégrité	Renforcée : seuls les services légitimes (22, 80) sont exposés
Disponibilité	Maintenue : HTTP et SSH restent accessibles pour les utilisateurs autorisés

Table 4 – Impact de la configuration iptables sur la triade CIA

Conclusion

Ce TP a permis de mettre en pratique la configuration d'un pare-feu Linux basé sur **iptables** dans un environnement réseau réel. Nous avons suivi une approche progressive : partir d'un état totalement fermé (`INPUT DROP`) puis ouvrir sélectivement les services nécessaires.

Points clés appris

- La politique **"tout bloquer par défaut, autoriser explicitement"** (whitelist) est la bonne pratique en sécurité réseau ;
- Les règles iptables sont appliquées **dans l'ordre** : la première règle correspondante l'emporte ;
- La règle `RELATED,ESTABLISHED` est indispensable pour maintenir les connexions TCP valides sans bloquer les réponses ;
- Le filtrage SSH par adresse source (`-s`) ajoute une couche de sécurité supplémentaire en limitant l'exposition du service ;
- Nmap est un outil puissant pour vérifier l'état réel du filtrage depuis la perspective d'un hôte externe ;

Mesures complémentaires de sécurité

Pour renforcer davantage la sécurité du serveur web :

- Utiliser `iptables-save` pour persister les règles après redémarrage ;
- Ajouter des règles de limitation de débit (`--limit`) contre les attaques par force brute sur SSH ;
- Activer `fail2ban` pour bloquer automatiquement les IP après plusieurs tentatives d'authentification échouées ;
- Considérer la migration vers `nftables`, le successeur moderne d'iptables dans les noyaux Linux récents ;